ព្រះរាជាណាចក្រកម្ពុជា

ជាតិ សាសនា ព្រះមហាក្សត្រ



សាកលវិទ្យាល័យ សៅស៍អ៊ីសថ៍អេយស៊ៀ

UNIVERSITY OF SOUTH-EAST ASIA

របាយការណ៍កិច្ចការស្រាវជ្រាវ

Assignment Report

ប្រធានបទ

Cloud Infrastructure & DevOps Automation Project

មុខវិជ្ជា	:	Advanced Cloud Technology
សាស្ត្រាចារ្យ	:	**លោកគ្រូ ក្រុល ឡឿន**
សមាជិកក្រុម	:	**១. និស្សិត សេង គឹមអូន**
		២. និស្សិត ខិន គឹមណាម
ឆ្នាំសិក្សា	:	**២០២៦ - ២០២៧**

១. សេចក្តីផ្តើម

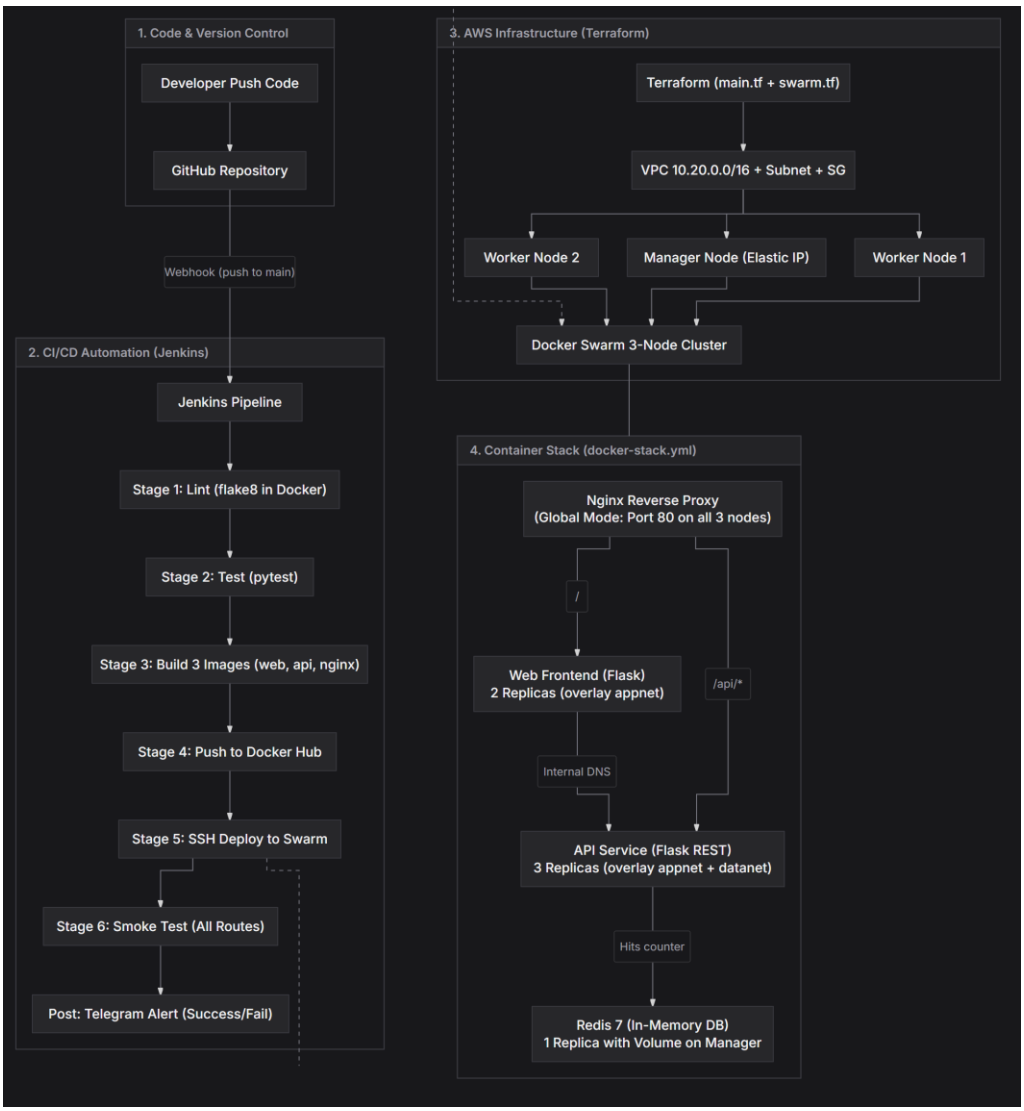
បច្ចេកវិទ្យា Cloud Computing និងវប្បធម៌ DevOps បាន ដើរ តួនាទីយ៉ាងសំខាន់មិនអាចខ្វះ បានក្នុងការស្រាវជ្រាវ ការអភិវឌ្ឍ ការដាក់ឱ្យដំណើរការ និងការគ្រប់គ្រងកម្មវិធីក្នុងបរិស្ថានឌីជីថល សម័យទំនើប។ បច្ចុប្បន្ននេះ ស្ថាប័នបច្ចេកវិទ្យា និងក្រុមហ៊ុនអភិវឌ្ឍន៍សូហ្វវែរនានា បានផ្លាស់ប្តូរ ទម្លាប់ពីការដំឡើងប្រព័ន្ធដោយដៃ (Manual Provisioning) មកកាន់ការអនុវត្តស្វ័យប្រវត្តិកម្ម ពេញលេញតាមរយៈគោលការណ៍ GitOps ដែលចាត់ទុកកូដជាប្រភពនៃការពិតតែមួយគត់ (Single Source of Truth)។

របាយការណ៍កិច្ចការស្រាវជ្រាវនេះ ផ្តោតសំខាន់លើការសិក្សា និងការរួមបញ្ចូលគ្នា យ៉ាងស៊ីជម្រៅរវាងហេដ្ឋារចនាសម្ព័ន្ធ Cloud របស់ Amazon Web Services (AWS) និងលំហូរ ការងារ DevOps Automation ដើម្បីកសាងប្រព័ន្ធកម្មវិធី Microservices ប្រកបដោយភាពធន់ខ្ពស់ (High Availability) សុវត្ថិភាពរឹងមាំ និងគ្មានការផ្អាកដំណើរការនៅពេល Update កំណែថ្មី (Zero-Downtime Deployment)។

ប្រព័ន្ធទាំងមូលត្រូវបានចងក្រង និងតភ្ជាប់គ្នាយ៉ាងស្អិតរមួតចាប់ពីដើមរហូតដល់ចប់ (End-to-End Workflow)៖

- **Git Version Control:** ការគ្រប់គ្រងប្រភពកូដ និងកំណែប្រែតាមរយៈ GitHub ដោយ ប្រើប្រាស់ Branching Strategy និង Pull Request Review
- **Infrastructure as Code (IaC):** ការកំណត់ និងបង្កើតហេដ្ឋារចនាសម្ព័ន្ធ Cloud ដោយ ស្វ័យប្រវត្តិតាមរយៈ HashiCorp Terraform (Infrastructure as Code - IaC) ដោយ បង្កើត VPC, Subnets, Security Groups, Elastic IP និង EC2 Instances ដោយមិន ចាំបាច់ចុចលើ Cloud Console ដោយដៃ
- **Containerization:** ការវេចខ្ចប់សេវាកម្មកម្មវិធីជា Container ដាច់ដោយឡែកពីគ្នាដោយ Docker និងការធ្វើតេស្តដំណើរការរួមគ្នានៅលើ Local Environment តាមរយៈ Docker Compose
- **Multi-Node Orchestration:** ការគ្រប់គ្រង និងចាត់ចែង Container នៅលើ Multi-Host Cluster ដោយប្រើ Docker Swarm (១ Manager Node និង ២ Worker Nodes) ដែលគាំទ្រ Ingress Routing Mesh និង Service VIPs

- **CI/CD Pipeline Automation:** ការសាងសង់ CI/CD Pipeline ស្វ័យប្រវត្តិពេញលេញ ៧ ដំណាក់កាល ដោយ Jenkinsfile ភ្ជាប់ GitHub Webhook សម្រាប់ Linting, Automated Pytest, Build, Push Docker Hub, Remote Deploy និង Smoke Test
- **Ingress Routing & Load Balancing:** ការរៀបចំ Nginx ជា Reverse Proxy & Load Balancer នៅ Port 80 សម្រាប់ទទួលសំណើពីអ្នកប្រើប្រាស់ និងចែកចាយចរាចរណ៍ទិន្នន័យទៅកាន់ Web Frontend និង API Backend
- **ChatOps Telemetry:** ការរួមបញ្ចូលប្រព័ន្ធ ChatOps តាមរយៈ Telegram Bot សម្រាប់ផ្ញើសារ Alert ជូនដំណឹងអំពីស្ថានភាព Build, Deploy និង Rollback ភ្លាមៗទៅកាន់ទូរស័ព្ទដៃរបស់ក្រុមការងារ។



រូបភាពទី ១.១៖ *ស្ថាបត្យកម្មប្រព័ន្ធរួម (End-to-End System Architecture: GitOps, Jenkins CI/CD, AWS Terraform & Docker Swarm)*

២. ឈំនាំបញ្ហា

ការអភិវឌ្ឍ និងការដាក់ឱ្យដំណើរការកម្មវិធីនៅលើ Cloud នាសម័យបច្ចុប្បន្ន ទាមទារឱ្យមានការសម្របសម្រួល និងសង្គតិភាពខ្ពស់រវាងប្រភពកូដ បរិស្ថានដំណើរការ បណ្តាញ និងម៉ាស៊ីនបម្រើ។ កន្លងមក វិធីសាស្ត្រប្រតិបត្តិការបែបប្រពៃណី (ClickOps ឬ Manual Operations) តែងតែជួបប្រទះនូវបញ្ហាប្រឈមធំៗជាច្រើនដូចជា៖

- **ភាពយឺតយ៉ាវ និងកំហុសដោយការចុចដៃ (Manual Console Bottlenecks):** ការបង្កើត Virtual Servers, បណ្តាញ VPC, និងការកំណត់ Firewall Security Groups តាមរយៈ Web Console ដោយដៃ ចំណាយពេលយូរ និងងាយបង្កឱ្យមានកំហុសអចេតនា (Human Error) ដូចជាការកំណត់ច្រឡំ Port ឬអនុញ្ញាតសិទ្ធិចូលជ្រុល
- **បញ្ហាលម្អៀងនៃការកំណត់រចនាសម្ព័ន្ធ (Configuration Drift):** នៅពេលបរិស្ថាន Local Development និងបរិស្ថាន Cloud Staging/Production មានការកំណត់ខុសគ្នា (Dependencies, Environment Variables, OS Packages) នាំឱ្យកើតមានបញ្ហា 'Works on my machine, but breaks on server' ដែលពិបាកស្វែងរកមូលហេតុ និងដោះស្រាយ
- **កង្វះប្រវត្តិ និងការតាមដានការផ្លាស់ប្តូរ (No Audit Trail & Versioning):** ការចុចបង្កើត ឬកែប្រែធនធាននៅលើ Cloud Console មិនមានកត់ត្រា Audit Trail ឬ Git Commit History នោះទេ ដែលធ្វើឱ្យបាត់បង់ដានការផ្លាស់ប្តូរ និងពិបាកសហការគ្នាក្នុងក្រុម
- **ការស្តារប្រព័ន្ធឡើងវិញមានភាពស្មុគស្មាញ (Painful Disaster Recovery):** នៅពេល Server គាំង ឬជួបឧប្បទ្ទវហេតុការបង្កើតប្រព័ន្ធឡើងវិញពីដំបូងតម្រូវឱ្យវិស្វករចងចាំដំហានរាប់សិបដំណាក់កាលក្រោមសម្ពាធ ដែលនាំឱ្យពេលវេលាប្រព័ន្ធគាំង (Downtime) អូសបន្លាយយូរ និងប៉ះពាល់ដល់អាជីវកម្ម
- **ភាពស្មុគស្មាញនៃ Multi-Node Orchestration:** ការគ្រប់គ្រង Microservices នៅលើម៉ាស៊ីនច្រើន (Multi-Node) ដោយដៃ មានការលំបាកក្នុងការធានាតុល្យភាពបន្ទុក (Load Balancing), Service Discovery និងការ Update កំណែកូដដោយមិនឱ្យដាច់ចរាចរណ៍ទិន្នន័យរបស់អ្នកប្រើប្រាស់ (Downtime)
- **ហានិភ័យនៃការលេចធ្លាយព័ត៌មានសម្ងាត់ (Secret Leaks):** ការភ្លេចខ្លួនដាក់ Credentials, SSH Keys, AWS Keys ឬ Telegram Bot Tokens នៅក្នុងកូដ

សាធារណៈ (Public GitHub) បង្កជាហានិភ័យសន្តិសុខធ្ងន់ធ្ងរដល់ទិន្នន័យ និងហេដ្ឋារចនាសម្ព័ន្ធ។

ផ្អែកលើបញ្ហាប្រឈមខាងលើ គម្រោងនេះត្រូវបានបង្កើតឡើងដើម្បីឆ្លើយតបទៅនឹងសំណួរស្រាវជ្រាវសំខាន់ថា៖ «តើយើងអាចរួមបញ្ចូលបច្ចេកវិទ្យា GitHub, Jenkins, Terraform, Docker, Docker Swarm, AWS EC2, Nginx និង Telegram យ៉ាងដូចម្តេច ដើម្បីកសាងលំហូរការងារស្វ័យប្រវត្តិកម្ម GitOps ដែលអាចទុកចិត្តបាន មានសុវត្ថិភាពខ្ពស់ និងធានាការដាក់ឱ្យដំណើរការប្រកបដោយ Zero-Downtime?»

៣. វត្ថុបំណងនៃគម្រោង

គម្រោងនេះមានគោលបំណងចម្បងក្នុងការសិក្សា អនុវត្ត និងកសាងហេដ្ឋារចនាសម្ព័ន្ធ Cloud និងលំហូរការងារ DevOps Automation ដែលតភ្ជាប់គ្រប់ដំណាក់កាលទាំងអស់ឱ្យក្លាយជាប្រព័ន្ធស្វ័យប្រវត្តិតែមួយ។ វត្ថុបំណងជាក់លាក់រួមមាន៖

- **ការគ្រប់គ្រងកំណែកូដ (Version Control):** រៀបចំរចនាសម្ព័ន្ធ Folder និងគ្រប់គ្រងប្រភពកូដតាមរយៈ Git & GitHub ដោយប្រើ Branching Strategy, Meaningful Commits និង Pull Request ដើម្បីធានាបាននូវ Audit Trail ពេញលេញ
- **ការវេចខ្ចប់ជា Container (Containerization):** សរសេរ Dockerfile ប្រកបដោយស្តង់ដារសម្រាប់សេវាកម្មនីមួយៗ (Web UI, REST API, Nginx Proxy) និងរៀបចំ Docker Compose សម្រាប់ធ្វើតេស្តរួមគ្នានៅលើ Local Environment មុនពេល Push ទៅកាន់ Docker Hub Registry។
- **ស្វ័យប្រវត្តិកម្មហេដ្ឋារចនាសម្ព័ន្ធ (Infrastructure as Code):** សរសេរកូដ Terraform (IaC) ដើម្បីបង្កើត Dedicated AWS VPC (10.20.0.0/16), Internet Gateway, Public Subnet, Least-Privilege Security Groups, Elastic IP និង EC2 Instances ចំនួន ៣ ដោយស្វ័យប្រវត្តិ ព្រមទាំងរក្សាទុក State File នៅលើ AWS S3 ជាមួយ Versioning
- **ការគ្រប់គ្រង Cluster និងបណ្តាញ (Swarm Orchestration):** ផ្តើមដំណើរការ Docker Swarm Cluster (១ Manager Node និង ២ Worker Nodes) ព្រមទាំងកំណត់

Overlay Networks (appnet និង datanet) ដើម្បីបំបែកស្រទាប់ទិន្នន័យ (Database Isolation) និងសម្រួលដល់ Service Discovery

- **ការចែកចាយបន្ទុក (Load Balancing):** កំណត់ Nginx ជា Reverse Proxy និង Ingress Load Balancer នៅ Port 80 លើគ្រប់ Node ដើម្បីទទួល និងចែកចាយសំណើទៅកាន់ Web Frontend (2 Replicas) និង REST API (3 Replicas) ដោយស្វ័យប្រវត្តិ
- **ការសាងសង់ CI/CD Pipeline (Continuous Integration & Deployment):** សរសេរ Jenkinsfile ៧ ដំណាក់កាល (Checkout → Lint → Test → Build → Push → Deploy → Smoke Test) ភ្ជាប់ GitHub Webhook ដើម្បីឱ្យការ Push កូដទៅ main ក្លាយជាការ Deploy ទៅ EC2 ដោយស្វ័យប្រវត្តិក្នុងរយៈពេលក្រោម ៩០ វិនាទី
- **ការធានានូវភាពធន់ និងគ្មាន Downtime (Zero-Downtime & Resiliency):** អនុវត្តយុទ្ធសាស្ត្រ Rolling Update តាមរយៈ 'order: start-first' និង 'parallelism: 1' ដើម្បីធានាថាការដាក់កំណែថ្មីមិនមានការធ្លាក់ចុះចរាចរណ៍ទិន្នន័យ (Zero Dropped Packets) ព្រមទាំងធ្វើតេស្តភាពធន់នៃ Node Failover
- **ការពង្រឹងសន្តិសុខពពក (Cloud Security Hardening):** អនុវត្តគោលការណ៍ Zero Committed Secrets ដោយប្រើប្រាស់ Jenkins Credentials Store, បិទ Password SSH (ប្រើ Key-Pair Only), បើកដំណើរការ AWS IMDSv2 និង Encrypted gp3 EBS Storage។
- **ប្រព័ន្ធតាមដាន ChatOps (Real-Time Telemetry):** រួមបញ្ចូល Telegram Bot API ក្នុង Pipeline Post Action ដើម្បីផ្ញើសារជូនដំណឹងភ្លាមៗ (☑ SUCCESS / ✖ FAILED) អមដោយព័ត៌មានលម្អិតនៃ Build និង Commit Link។
- **ការផ្ទៀងផ្ទាត់តាមភស្តុតាង (Empirical Verification):** ផ្ទៀងផ្ទាត់លទ្ធផលជាក់ស្តែងតាមរយៈរូបភាព Screenshots ចំនួន ១៦+ សន្លឹក និងការតេស្តកូដជាក់ស្តែង (Smoke Test) ដើម្បីបញ្ជាក់ពីភាពជោគជ័យតាម Rubric ពិន្ទុរបស់សាកលវិទ្យាល័យ។

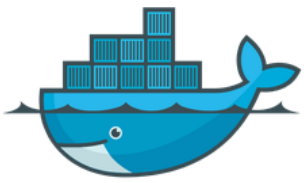
៤. បច្ចេកវិទ្យាដែលប្រើក្នុងគម្រោង

ដើម្បីសម្រេចបាននូវប្រព័ន្ធស្វ័យប្រវត្តិកម្ម GitOps ប្រកបដោយប្រសិទ្ធភាពខ្ពស់ គម្រោងនេះបានជ្រើសរើស និងរួមបញ្ចូលគ្នានូវបច្ចេកវិទ្យាស្នូលឈានមុខគេក្នុងឧស្សាហកម្ម Cloud & DevOps ចំនួន ៨ ដូចខាងក្រោម៖

៤.១- Docker & Docker Compose

Docker គឺជាបច្ចេកវិទ្យា Containerization កម្រិត Operating-System-Level Virtualization ដែលអនុញ្ញាតឱ្យយើងវេចខ្ចប់កម្មវិធី (Application) ព្រមទាំង Dependencies, Libraries និង Configuration Files ទាំងអស់ឱ្យក្លាយទៅជា Docker Image តែមួយដែលអាចយកទៅដំណើរការបានគ្រប់ទីកន្លែងដោយភាពស៊ីសង្វាក់គ្នា។

- Docker Image: ជា Blueprint ឬ Template សម្រាប់បង្កើត Container ដែលមានលក្ខណៈ Immutable (មិនអាចកែប្រែក្រោយពេល Build)
- Docker Container: ជា Instance ជាក់ស្តែងដែលកំពុងដំណើរការដោយឯករាជ្យ និងដាច់ដោយឡែកពីប្រព័ន្ធប្រតិបត្តិការ Host (Isolated User Space)
- Dockerfile: ជាឯកសារអត្ថបទដែលផ្ទុកនូវបណ្តុំ Command សម្រាប់បង្កើត Docker Image តាមលំដាប់លំដោយ
- Docker Compose: ជាឧបករណ៍សម្រាប់កំណត់ និងដំណើរការកម្មវិធីដែលមាន Multi-Container នៅលើបរិស្ថាន Local ដោយប្រើឯកសារ docker-compose.yml តែមួយ
- Docker Hub Registry: ជាសេវាកម្ម Cloud Repository សម្រាប់រក្សាទុក និងចែករំលែក Docker Images ជាសាធារណៈ ឬឯកជន។



រូបភាពទី ៤.១៖ *Logo បច្ចេកវិទ្យា Docker*

៤.២- HashiCorp Terraform (Infrastructure as Code - IaC)

Terraform គឺជាឧបករណ៍ Open-source Infrastructure as Code (IaC) ដ៏មានឥទ្ធិពលបំផុត ដែលប្រើប្រាស់ HashiCorp Configuration Language (HCL) ដើម្បីកំណត់ និងបង្កើត

ធនធាន Cloud (VPC, Subnets, Security Groups, EC2) តាមរបៀប Declarative (កំណត់លទ្ធផលចុងក្រោយដែលចង់បាន)។

- terraform init: ផ្តើមរៀបចំ Working Directory និងទាញយក AWS Provider Plugins
- terraform plan: ពិនិត្យ និងបង្ហាញ Diff រវាងស្ថានភាពជាក់ស្តែងនៅលើ AWS និងកូដក្នុង Configuration មុនពេលអនុវត្ត
- terraform apply: បង្កើត ឬកែប្រែធនធានទាំងអស់នៅលើ AWS Cloud ដោយស្វ័យប្រវត្តិ
- terraform destroy: លុបបំបាត់ និងសម្អាតធនធានទាំងអស់ដែលបានបង្កើតដោយ Terraform យ៉ាងស្អាតដោយមិនបន្សល់ទុកកាកសំណល់
- S3 Remote State Backend: រក្សាទុកឯកសារ terraform.tfstate នៅលើ AWS S3 Bucket ជាមួយ Versioning Enabled ដើម្បីការពារការបាត់បង់ទិន្នន័យ និងសម្រួលដល់ការងារជាក្រុម។



រូបភាពទី ៤.២៖ *Logo បច្ចេកវិទ្យា HashiCorp Terraform*

៤.៣- Git & GitHub

Git គឺជាប្រព័ន្ធ Distributed Version Control System (VCS) ដែលតាមដានរាល់ការផ្លាស់ប្តូរកូដ ខណៈ GitHub គឺជា Cloud-based Hosting Platform សម្រាប់ផ្ទុកកូដ គ្រប់គ្រងកិច្ចសហការ និងធ្វើស្វ័យប្រវត្តិកម្ម CI/CD តាមរយៈ Webhooks។

- Git Repository: កន្លែងផ្ទុកប្រភពកូដ, IaC manifests, Dockerfiles និង configuration files ទាំងអស់
- Commit History: កត់ត្រារាល់ការកែប្រែជាមួយសារពន្យល់ច្បាស់លាស់ (Meaningful Commit Messages)
- GitHub Webhook: ធ្វើ HTTP POST Request ស្វ័យប្រវត្តិតាមរយៈ Event 'push' ទៅកាន់ Jenkins Server ដើម្បីដាស់ Pipeline ឱ្យចាប់ផ្តើមដំណើរការភ្លាមៗ។



រូបភាពទី ៤.៣៖ *Logo បច្ចេកវិទ្យា Git & GitHub*

៤.៤- Jenkins CI/CD Automation

Jenkins គឺជា Automation Server ឈានមុខគេដែលគាំទ្រស្ថាបត្យកម្ម Pipeline as Code តាមរយៈឯកសារ Jenkinsfile។ Jenkins ជួយលុបបំបាត់ដំហាន Deploy ដោយដៃទាំងស្រុង និងអនុវត្ត Quality Gates យ៉ាងម៉ត់ចត់។

- Continuous Integration (CI): ធ្វើការទាញយកកូដថ្មី (Checkout), ដំណើរការ Flake8 Linting ក្នុង Container និងដំណើរការ Automated Pytest Unit Tests គ្រប់ពេលដែលមាន Commit ថ្មី
- Continuous Delivery / Deployment (CD): ធ្វើការ Build Docker Images, Push ទៅកាន់ Docker Hub និង SSH ទៅកាន់ AWS EC2 Manager Node ដើម្បីដាក់ពង្រាយ (Deploy) កំណែថ្មីដោយស្វ័យប្រវត្តិ
- Credentials Management: រក្សាទុក Secret Keys, SSH Private Keys, Docker Hub Token និង Telegram Token ក្នុងទម្រង់ Masked Credentials ដោយមិនសរសេរចូលក្នុងកូដជាដាច់ខាត។

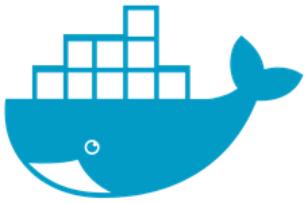


រូបភាពទី ៤.៤៖ *Logo បច្ចេកវិទ្យា Jenkins Automation Server*

៤.៥- Docker Swarm Multi-Node Orchestration

Docker Swarm គឺជាមុខងារ Native Clustering & Orchestration ដែលភ្ជាប់មកជាមួយ Docker Engine ស្រាប់ សម្រាប់គ្រប់គ្រង និងចាត់ចែង Services នៅលើ Cluster នៃម៉ាស៊ីនបម្រើច្រើន (Multi-Host)។

- Manager Node: គ្រប់គ្រងស្ថានភាព Cluster (Raft Consensus), ទទួល Stack Manifests និងកំណត់ភារកិច្ចទៅកាន់ Workers
- Worker Nodes: ទទួល និងដំណើរការ Tasks/Containers តាមការចាត់តាំងរបស់ Manager
- Ingress Routing Mesh: អនុញ្ញាតឱ្យចរាចរណ៍ទិន្នន័យចូលមកកាន់ Port 80 នៃម៉ាស៊ីន EC2 ណាមួយក៏បាន ហើយ Swarm នឹងបញ្ជូនទៅកាន់ Container ដែលកំពុងដំណើរការយ៉ាងត្រឹមត្រូវ
- Overlay Networks: បណ្តាញនិម្មិត VXLAN ដែលភ្ជាប់ Containers ឆ្លងកាត់ Host ផ្សេងៗគ្នា (appnet សម្រាប់ Frontend/API និង datanet សម្រាប់ API/Redis)
- Service VIP Load Balancing: Docker DNS ផ្តល់នូវ Virtual IP សម្រាប់ Service នីមួយៗ និងចែកចាយ Request ទៅកាន់ Replicas ទាំងអស់តាមយន្តការ Round-Robin។



រូបភាពទី ៤.៥៖ *Logo បច្ចេកវិទ្យា Docker Swarm*

៤.៦- Amazon Web Services (AWS EC2 & VPC)

Amazon Web Services (AWS) គឺជា Cloud Provider ដ៏ធំបំផុតនៅលើពិភពលោក។ នៅក្នុងគម្រោងនេះ សេវាកម្ម AWS ស្នូលចំនួនពីរត្រូវបានប្រើប្រាស់រួមមាន Amazon Virtual Private Cloud (VPC) និង Amazon Elastic Compute Cloud (EC2)។

- Custom VPC (10.20.0.0/16): បណ្តាញពពកដាច់ដោយឡែក ដែលមាន Internet Gateway និង Public Subnet សម្រាប់ធានាសុវត្ថិភាពទិន្នន័យ

- EC2 Instances: Virtual Servers ដំណើរការលើប្រព័ន្ធប្រតិបត្តិការ Ubuntu 24.04 LTS (១ Manager Node និង ២ Worker Nodes)
- Elastic IP (EIP): អាសយដ្ឋាន Public IP ថេរមួយដែលត្រូវបានភ្ជាប់ជាមួយ Manager Node សម្រាប់ឱ្យ Jenkins SSH ចូល និងអ្នកប្រើប្រាស់ចូលមើលគេហទំព័រ។
- Security Groups: Virtual Firewalls ដែលអនុញ្ញាតតែ Port ចាំបាច់ (Port 22 សម្រាប់ Admin SSH, Port 80 សម្រាប់ HTTP Web, និង Swarm Ports 2377 / 7946 / 4789 សម្រាប់តែផ្ទៃក្នុង Cluster)។



រូបភាពទី ៤.៦៖ *Logo សេវាកម្ម Amazon Web Services (AWS)*

៤.៧- Nginx Reverse Proxy & Load Balancer

Nginx គឺជា High-Performance Web Server និង Reverse Proxy ដែលដើរតួជាច្រកចេញចូលចម្បង (Ingress Entrypoint) នៅ Port 80 សម្រាប់ទទួលសំណើពីអ្នកប្រើប្រាស់ និងបញ្ជូនបន្តទៅកាន់ Services ខាងក្រោយ។ Nginx ត្រូវបានកំណត់ឱ្យដំណើរការក្នុង 'mode: global' (មាន ១ Container នៅលើគ្រប់ EC2 Node) និងប្រើប្រាស់ Embedded Docker DNS Resolver (127.0.0.11) ដើម្បីស្វែងរក Service VIPs របស់ Docker Swarm ដោយស្វ័យប្រវត្តិ។

៤.៨- សន្តិសុខពពក និងការគ្រប់គ្រង Secret (Cloud Security)

សន្តិសុខត្រូវបានអនុវត្តជាគោលការណ៍ចម្បង (Security by Design) ដោយធានាថាគ្មានលេខកូដសម្ងាត់ ឬ Private Keys ណាមួយត្រូវបាន Commit ចូលទៅក្នុង Git ឡើយ។ លើសពីនេះ ប្រព័ន្ធបានអនុវត្ត Token-based AWS IMDSv2 ដើម្បីទប់ស្កាត់ការវាយប្រហារ SSRF, បើកដំណើរការ Encryption At Rest លើ AWS EBS gp3 Storage, និងបិទ Password Login លើ EC2 ដោយអនុញ្ញាតតែ SSH Key-pair ប៉ុណ្ណោះ។

៤.៩- ប្រព័ន្ធតាមដាន ChatOps តាម Telegram Bot

Telegram ChatOps អនុញ្ញាតឱ្យវិស្វកររតាមដានស្ថានភាពប្រព័ន្ធបានគ្រប់ពេលវេលា និងគ្រប់ទីកន្លែង។ នៅពេល Jenkins Pipeline បញ្ចប់ដំណើរការ Jenkins នឹងហៅ Telegram Bot

API ភ្លាមៗដើម្បីផ្ញើសារសង្ខេបអំពីលទ្ធផល (☑ SUCCESS ឬ ✖ FAILED) អមដោយឈ្មោះ Job, លេខ Build, Git Commit Tag និង Link សម្រាប់ចូលពិនិត្យ Console Output។



រូបភាពទី ៤.៧៖ *Logo កម្មវិធី Telegram សម្រាប់ប្រព័ន្ធ ChatOps*

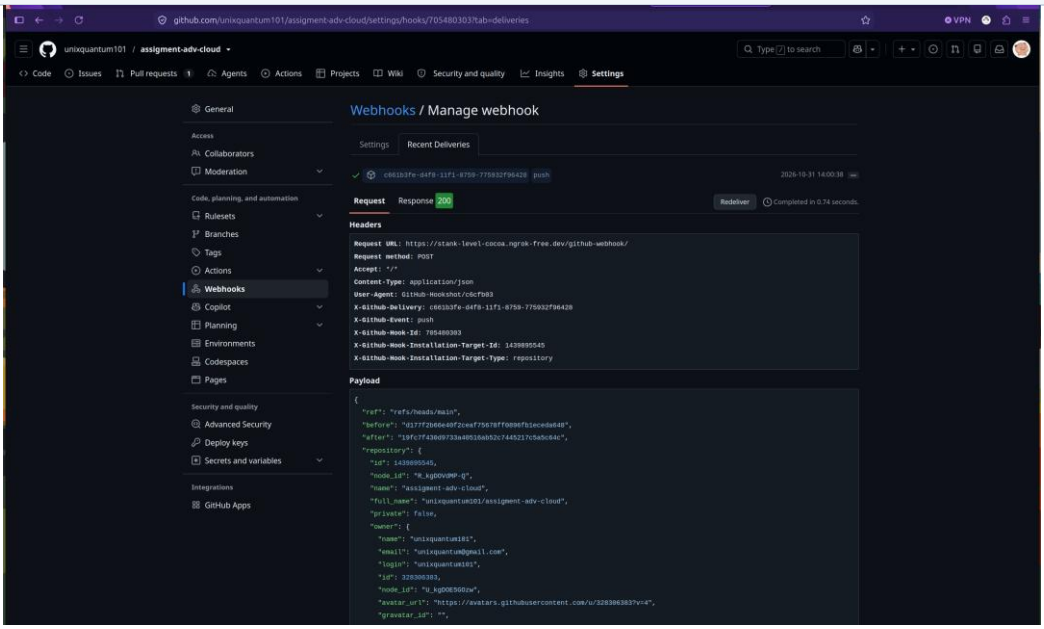
៥. របៀប Config តាមដំណាក់កាលអនុវត្ត

ផ្នែកនេះបង្ហាញពីដំណើរការអនុវត្តជាក់ស្តែងចំនួន ១២ ជំហាន ចាប់តាំងពីការរៀបចំកូដនៅលើ Local រហូតដល់ការដាក់ឱ្យដំណើរការលើ AWS EC2 Cloud ប្រកបដោយជោគជ័យ។

ជំហានទី ១: រៀបចំ Project Structure និង GitHub Repository

រៀបចំរចនាសម្ព័ន្ធ Folder ឱ្យមានរបៀបរៀបរយ ដោយបែងចែករវាង Microservices (Web, API), Nginx Reverse Proxy, Terraform IaC, Jenkinsfile, និង Scripts ព្រមទាំងបង្កើតឯកសារ .gitignore ដើម្បីការពារកុំឱ្យ Commit ព័ត៌មានសម្ងាត់ ឬឯកសារបណ្តោះអាសន្ន។

```
git init
git add .
git commit -m "feat: complete GitOps microservices capstone project"
git branch -M main
git remote add origin https://github.com/unixquantum101/assigment-adv-cloud.git
git push -u origin main
```



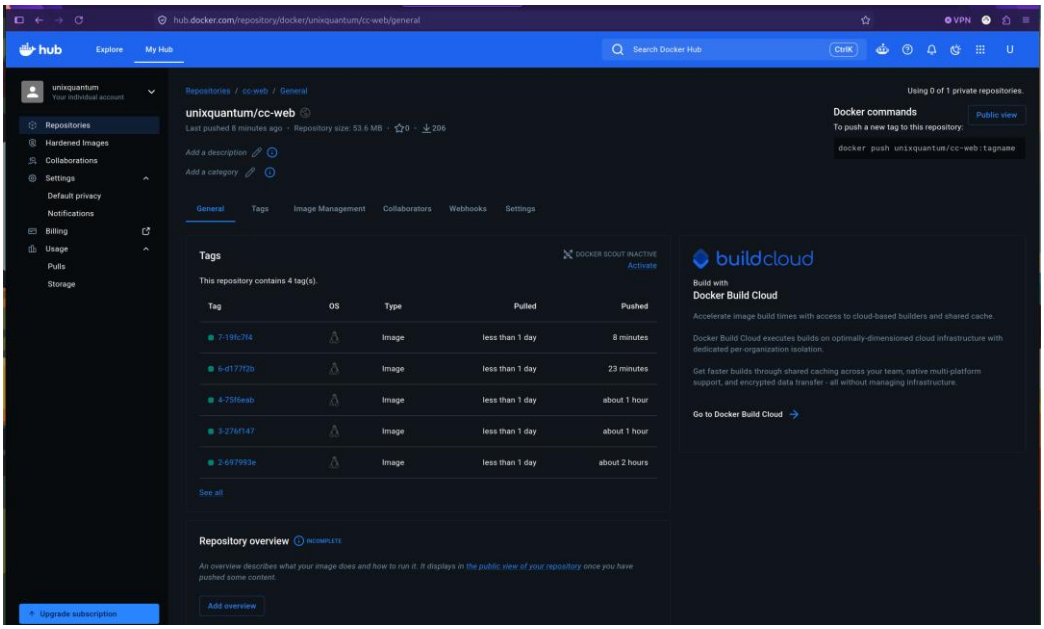
រូបភាពទី ៥.១៖ *ការរៀបចំ Repository និងការកំណត់ Webhook នៅលើ GitHub*

ជំហានទី ២: ការវេចខ្ចប់កម្មវិធីជា Docker Images

បង្កើត Dockerfile សម្រាប់សេវាកម្មនីមួយៗ ដោយប្រើប្រាស់ Base Image ដែលមានទំហំស្រាល (python:3.12-slim និង nginx:alpine) ព្រមទាំងចាក់បញ្ចូល Build Argument (APP_VERSION) ដើម្បីឱ្យ Container បង្ហាញលេខកំណែកូដយ៉ាងច្បាស់លាស់។

```
# Build and tag images locally
docker build -t unixquantum/cc-api:latest -t unixquantum/cc-api:14-a3f12c8
services/api
docker build -t unixquantum/cc-web:latest -t unixquantum/cc-web:14-a3f12c8
services/web
docker build -t unixquantum/cc-nginx:latest -t unixquantum/cc-nginx:14-a3f12c8 nginx

# Push images to Docker Hub Registry
docker push unixquantum/cc-api:14-a3f12c8
docker push unixquantum/cc-web:14-a3f12c8
docker push unixquantum/cc-nginx:14-a3f12c8
```



រូបភាពទី ៥.២៖ *Docker Hub Repositories ផ្ទុក Images ជាមួយ Traceable Semantic Tags*

ជំហានទី ៣: ការធ្វើតេស្ត Local Environment ដោយប្រើ Docker Compose

មុនពេលដាក់ពង្រាយទៅកាន់ Cloud ប្រព័ន្ធទាំងមូលត្រូវបានធ្វើតេស្តរួមគ្នានៅលើម៉ាស៊ីន Local តាមរយៈ docker-compose.yml ដើម្បីធ្វើឱ្យដំណើរការតភ្ជាប់រវាង Web Frontend, API Backend, Redis Datastore និង Nginx។

```
# Run full microservices stack locally
docker compose up --build -d

# Verify running containers and health
docker compose ps
curl http://localhost:8080/
curl http://localhost:8080/api/info
```



រូបភាពទី ៥.៣៖ *ការផ្ទៀងផ្ទាត់ដំណើរការ Local Development តាមរយៈ Docker Compose*

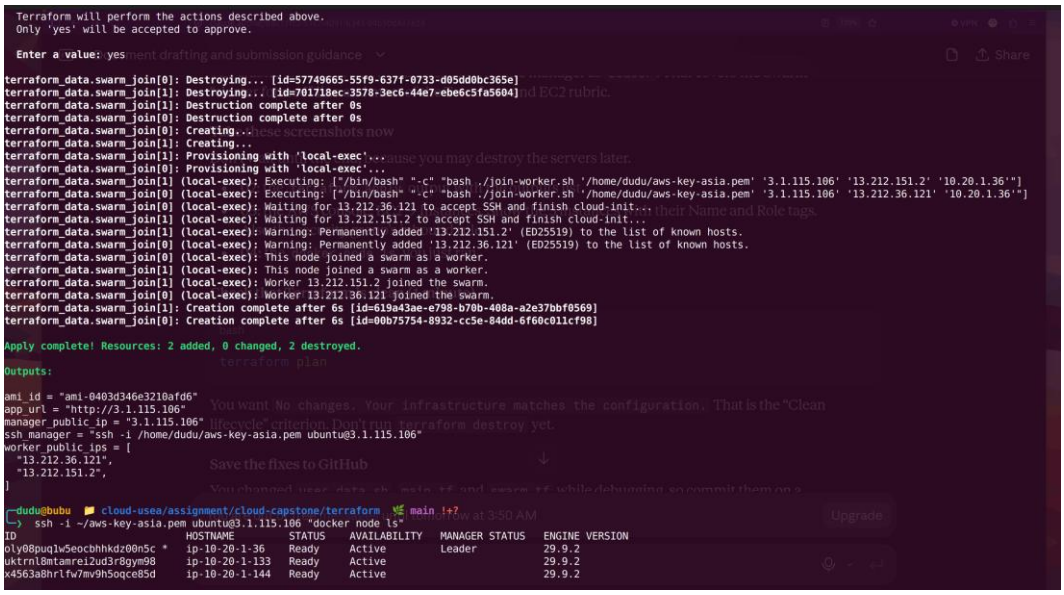
ជំហានទី ៤: ការសរសេរកូដ Terraform សម្រាប់ AWS Cloud

កំណត់រចនាសម្ព័ន្ធ Cloud ទាំងមូលនៅក្នុង Folder terraform/ រួមមាន main.tf (VPC & Subnets), security-group.tf, ec2.tf (1 Manager + 2 Workers), backend.tf (S3 Remote State) និង outputs.tf។

```
cd terraform
# Initialize remote backend and provider plugins
terraform init -backend-config="bucket=cc-tf-state-bucket" -backend-
config="region=ap-southeast-1"

# Review execution plan
terraform plan

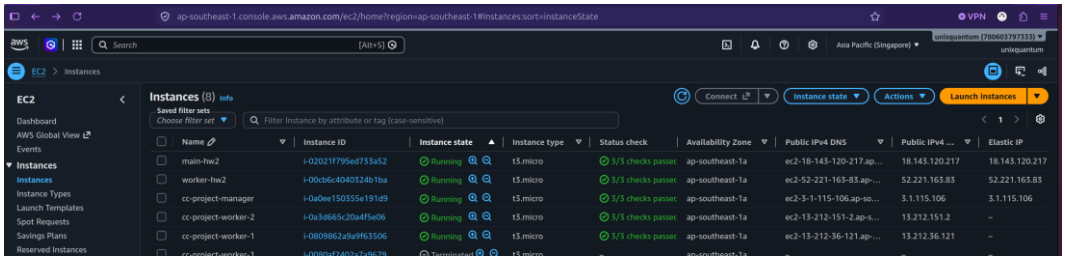
# Apply infrastructure changes to AWS
terraform apply -auto-approve
```



រូបភាពទី ៥.៤៖ *ការដំណើរការ 'terraform apply' បង្កើតហេដ្ឋារចនាសម្ព័ន្ធ AWS ទាំងស្រុង*

ជំហានទី ៥: ការផ្ទៀងផ្ទាត់ AWS EC2 Instances និង Security Groups

ពិនិត្យមើលលទ្ធផលនៅលើ AWS Management Console ដើម្បីបញ្ជាក់ថា EC2 Instances ទាំង៣ (cc-project-manager, cc-project-worker-1, cc-project-worker-2) កំពុងដំណើរការក្នុងស្ថានភាព 2/2 checks passed និងមានភ្ជាប់ Elastic IP ត្រឹមត្រូវ។



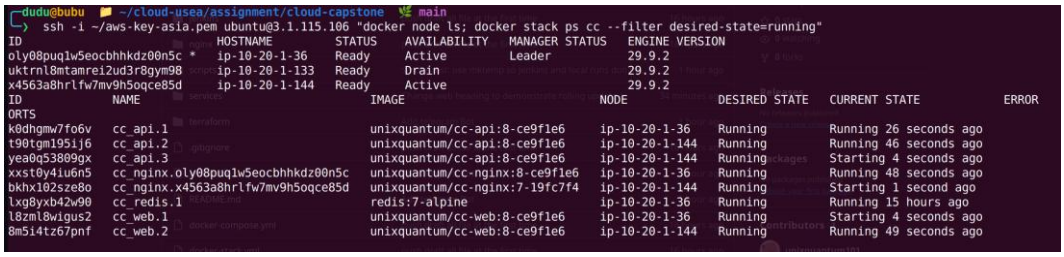
រូបភាពទី ៥.៥៖ *ផ្ទៀងផ្ទាត់លើ AWS EC2 Management Console បង្ហាញ Instances ទាំង ៣កំពុងដំណើរការ*

ជំហានទី ៦: ការបង្កើត Docker Swarm Multi-Node Cluster

ផ្តើមបង្កើត Swarm នៅលើ Manager Node តាមរយៈ 'docker swarm init' និងទាញយក Join Token ដើម្បីឱ្យ Worker Nodes ទាំងពីរចូលរួម (Join) បង្កើតបានជា 3-Node Cluster។

```
# SSH to Manager and check cluster membership
ssh -i ~/.ssh/cc-project ubuntu@3.1.115.106 "docker node ls"

# Output shows:
# ID            HOSTNAME          STATUS    AVAILABILITY    MANAGER STATUS    ENGINE
VERSION
# oly08puq1w *  ip-10-20-1-36     Ready     Active          Leader            29.9.2
# uktrn18mta    ip-10-20-1-133    Ready     Active                            29.9.2
# x4563a8hrl    ip-10-20-1-144    Ready     Active                            29.9.2
```



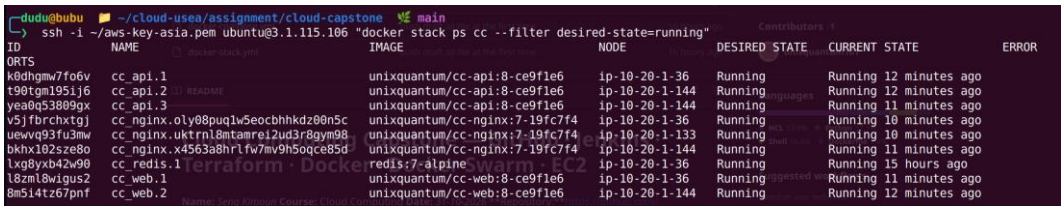
រូបភាពទី ៥.៦៖ *ការផ្ទៀងផ្ទាត់ Swarm Cluster តាមរយៈ 'docker node ls' (១ Manager Leader + ២ Workers)*

ជំហានទី ៧: ការរៀបចំ docker-stack.yml និងបណ្តាញ Overlay

រៀបចំឯកសារ docker-stack.yml ដែលកំណត់សេវាកម្មចំនួន ៤, ចំនួន Replicas, កម្រិត Resource Limits (CPU: 0.50, RAM: 192M), Restart Policies និងបណ្តាញ Overlay Networks ពីរដាន់៖ 'appnet' (ភ្ជាប់ Nginx, Web, API) និង 'datanet' (ភ្ជាប់ API, Redis ដោយឯកោ)។

```
# Deploy stack to Swarm
ssh -i ~/.ssh/cc-project ubuntu@3.1.115.106 \
  "TAG=14-a3f12c8 docker stack deploy -c docker-stack.yml cc --with-registry-auth"
```

```
# Verify task distribution across all 3 nodes
ssh -i ~/.ssh/cc-project ubuntu@3.1.115.106 "docker stack ps cc"
```



រូបភាពទី ៥.៧៖ *ការរៀងផ្ទាត់ 'docker stack ps cc' បង្ហាញ Tasks ចែកចាយស្មើគ្នាលើ EC2 Nodes ទាំង ៣*

ជំហានទី ៨: ការកំណត់ Nginx ជា Reverse Proxy & Load Balancer

Nginx ទទួល Traffic នៅ Port 80 ហើយធ្វើការ Proxy ទៅកាន់ Web Frontend (/) និង REST API (/api/*)។ នៅពេលធ្វើការ Request ទៅកាន់ /api/info ចំនួន ៣ ដង Swarm Routing Mesh នឹងបញ្ជូនទៅកាន់ API Replicas ផ្សេងៗគ្នាដែលបង្ហាញតាមរយៈ Hostname បង្វិលជុំ (Round-Robin)។

```
# Test Round-Robin Hostname Rotation
for i in {1..3}; do curl -s http://3.1.115.106/api/info | grep hostname; done

# Output proves load balancing across 3 different containers:
# {"hostname":"3189885337d9","service":"api","version":"8-ce9f1e6"}
# {"hostname":"d128da8a4e74","service":"api","version":"8-ce9f1e6"}
# {"hostname":"3189885337d9","service":"api","version":"8-ce9f1e6"}
```



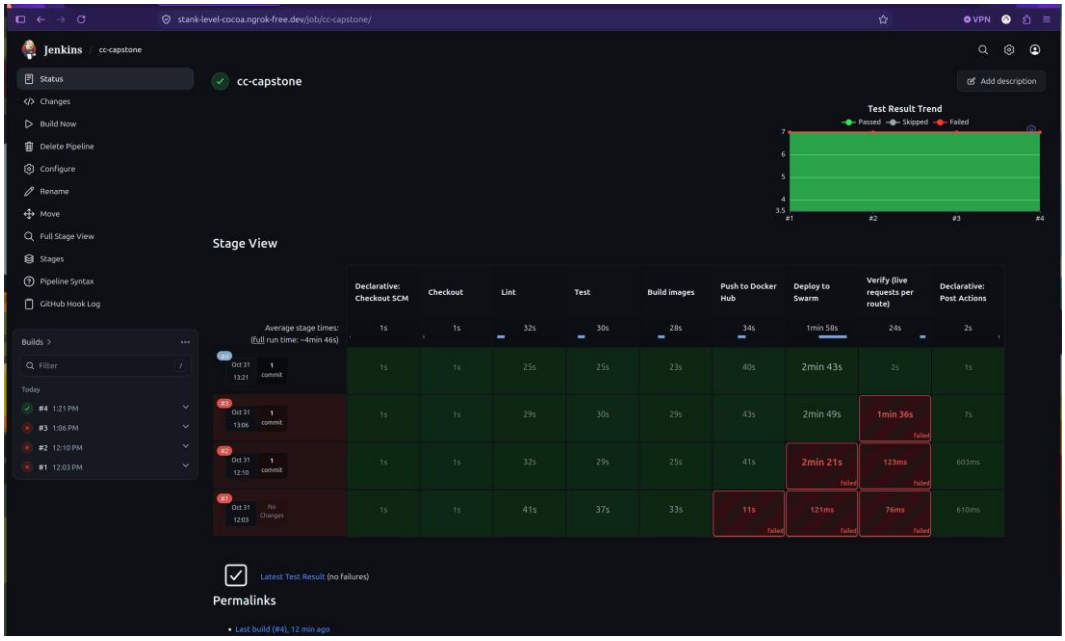
រូបភាពទី ៥.៨៖ *ភស្តុតាងជាក់ស្តែងនៃ Round-Robin Load Balancing បង្វិល Hostname លើ API Replicas*

ជំហានទី ៩: ការភ្ជាប់ GitHub Webhook ជាមួយ Jenkins

នៅលើ GitHub Repository កំណត់ Webhook ទៅកាន់ URL របស់ Jenkins (តាមរយៈ ngrok tunnel) ដោយជ្រើសរើស Content type 'application/json' និង Trigger Event 'Just the push event'។ នៅពេល Commit កូដថ្មី GitHub នឹងផ្ញើ HTTP 200 OK Response ទៅកាន់ Jenkins ភ្លាមៗ។

ជំហានទី ១០: ការកំណត់ 7-Stage Declarative Jenkins Pipeline

Jenkinsfile ត្រូវបានរៀបចំជា ៧ ដំណាក់កាលស្វ័យប្រវត្តិតាមលំដាប់លំដោយ៖ Checkout (បង្កើត Tag) → Lint (flake8 ក្នុង Python Container) → Test (pytest + JUnit XML) → Build Images → Push Docker Hub → Deploy Swarm → Smoke Test Verify។ ប្រសិនបើមាន Test ណាមួយបរាជ័យ Pipeline នឹងបញ្ឈប់ជាបន្ទាន់ (Pipeline Abort)។

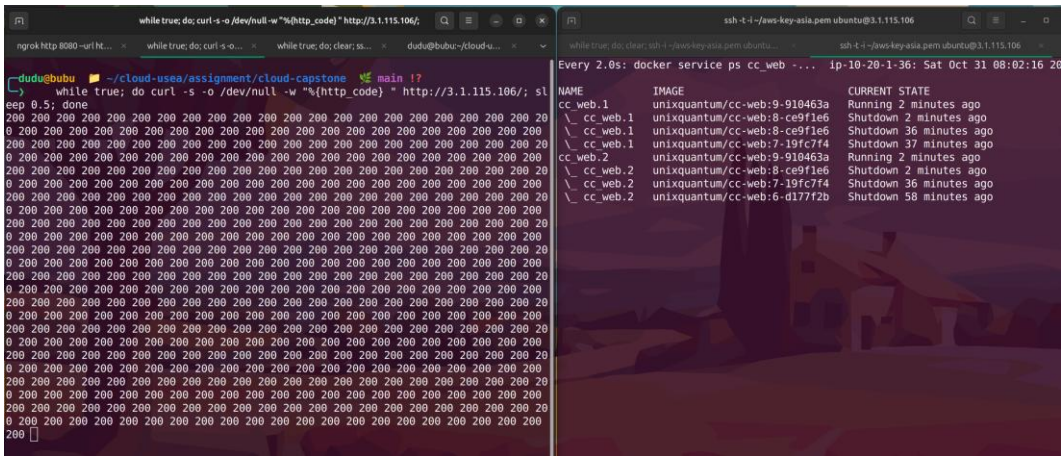


រូបភាពទី ៥.៩៖ *Jenkins Pipeline Stage View បង្ហាញគ្រប់ ៧ ដំណាក់កាលទទួលបានជោគជ័យបៃតង*

ជំហានទី ១១: ការអនុវត្ត Zero-Downtime Rolling Update និង Failover

នៅពេលមានការ Push កូដកំណែថ្មី Swarm នឹងអនុវត្តយុទ្ធសាស្ត្រ Rolling Update តាមរយៈ 'order: start-first' និង 'parallelism: 1' ដែលមានន័យថា Swarm នឹងបង្ហោះ Container ថ្មីឱ្យរត់ និងឆ្លងកាត់ Healthcheck សិន ទើបពន្លត់ Container ចាស់ជាបន្តបន្ទាប់។ ក្នុងអំឡុងពេលនេះការរត់ Curl Loop បង្ហាញថា គេហទំព័រឆ្លើយតប HTTP 200 OK ជាបន្តបន្ទាប់ដោយគ្មានការធ្លាក់ចុះចរាចរណ៍ទិន្នន័យឡើយ។

```
# Background continuous traffic test during rolling deploy
while true; do curl -s -o /dev/null -w "%{http_code}\n" http://3.1.115.106/; sleep
0.5; done
# 200
# 200
# 200 (Continuous 100% uptime, zero 502 Bad Gateway)
```

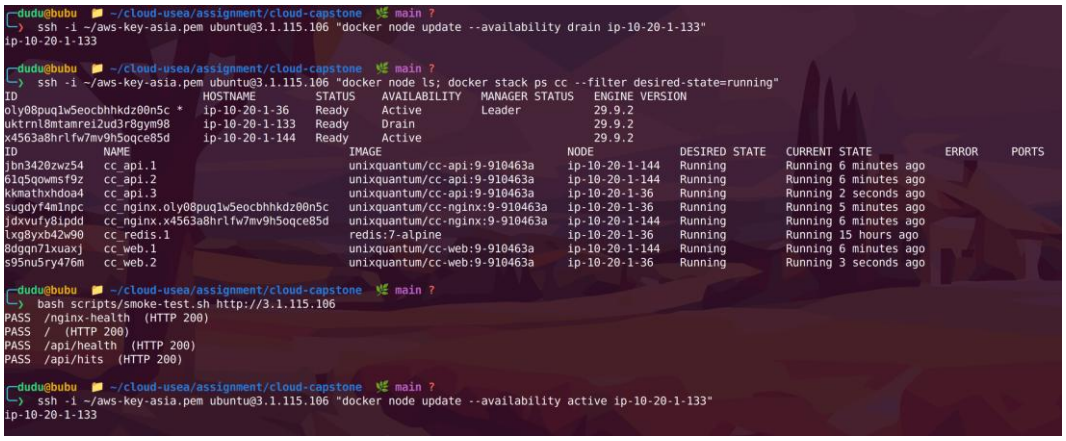


រូបភាពទី ៥.១០៖ *កស្តុតាងជាក់ស្តែងនៃ Rolling Update (Tasks ថ្មី Running មុនពេល Tasks ចាស់ Shutdown)*

ដើម្បីធ្វើតេស្តភាពធន់នៃប្រព័ន្ធនៅពេលមាន Server គាំង យើងបានធ្វើការ Drain Worker Node (worker-1)។ ជាលទ្ធផល Swarm បានរុះរើ និងបង្កើត Tasks ឡើងវិញនៅលើ Nodes ផ្សេងទៀតដោយស្វ័យប្រវត្តិក្នុងរយៈពេលក្រោម ៥ វិនាទី។

```
# Simulate worker failure by draining node
ssh -i ~/.ssh/cc-project ubuntu@3.1.115.106 \
  "docker node update --availability drain ip-10-20-1-133"

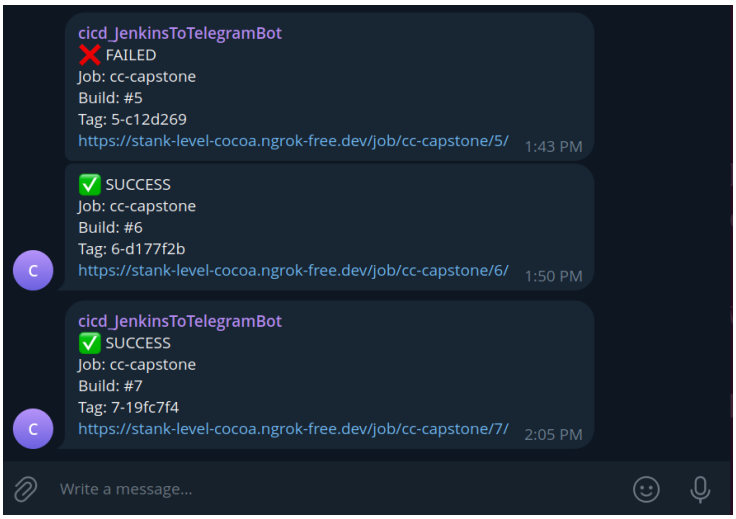
# Verify automatic rescheduling on remaining healthy nodes
ssh -i ~/.ssh/cc-project ubuntu@3.1.115.106 "docker stack ps cc"
```



រូបភាពទី ៥.១១៖ *កស្តុតាងនៃ Node Failover: Tasks ត្រូវបានរុះរើទៅកាន់ Nodes ផ្សេងដោយស្វ័យប្រវត្តិ*

ជំហានទី ១២: ការកំណត់ការជូនដំណឹងស្វ័យប្រវត្តិតាម Telegram Bot

បង្កើត Telegram Bot តាមរយៈ @BotFather និងយក Bot Token និង Chat ID ទៅរក្សាទុកក្នុង Jenkins Credentials Store ('telegram-bot-token', 'telegram-chat-id')។ នៅក្នុងប្លុក 'post { always }' នៃ Jenkinsfile ប្រព័ន្ធនឹងផ្ញើសារ Telegram ស្វ័យប្រវត្តិបង្ហាញស្ថានភាពជោគជ័យ (☑ SUCCESS) ឬបរាជ័យ (✖ FAILED)។



រូបភាពទី ៥.១២៖ *កស្តុតាងជាក់ស្តែងនៃសារជូនដំណឹង Telegram Bot Alerts នៅលើទូរស័ព្ទដៃ*

៦. លទ្ធផល និងការធ្វើតាមដាន់ប្រព័ន្ធ

លទ្ធផលនៃការអនុវត្តគម្រោងត្រូវបានកត់ត្រា និងធ្វើតាមដាន់យ៉ាងម៉ត់ចត់តាមរយៈកស្តុតាងជាក់ស្តែង ដើម្បីបញ្ជាក់ថាប្រព័ន្ធដំណើរការបានពេញលេញ ១០០% ស្របតាមលក្ខខណ្ឌវិនិច្ឆ័យ (Course Rubric) របស់សាកលវិទ្យាល័យ៖

១. ធ្វើតាមដាន់ Live Web Application UI & Redis Persistence: គេហទំព័រផ្សាយផ្ទាល់ដំណើរការយ៉ាងរលូននៅអាសយដ្ឋាន http://3.1.115.106/។ រាល់ពេល Refresh ទំព័រ ចំនួន Visits Counter ត្រូវបានកើនឡើងម្តងមួយៗ (Atomic Increment) ដោយទិន្នន័យត្រូវបានរក្សាទុកយ៉ាងរឹងមាំនៅក្នុង Redis Datastore តាមរយៈ Docker Named Volume (redis_data)។



រូបភាពទី ៦.១៖ *ធ្វើតាមដាន់ចំណុចប្រទាក់ Live Web Application UI (Cloud Computing Project v2)*

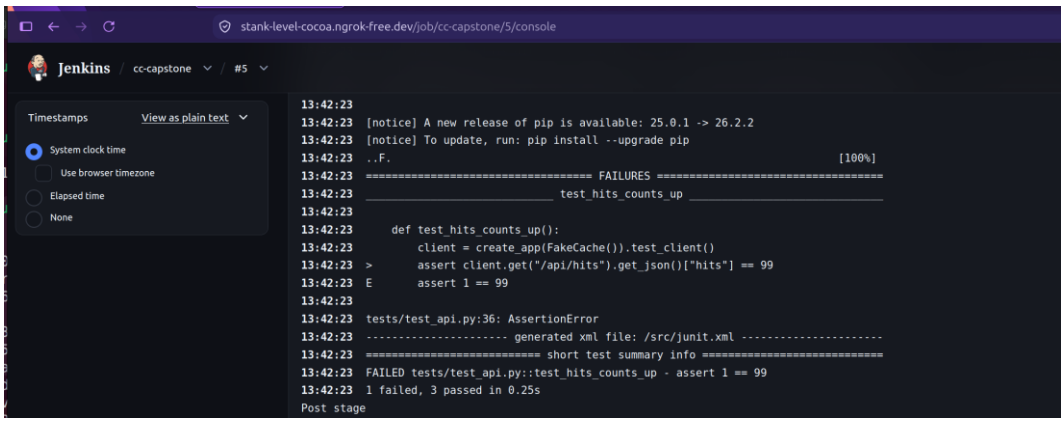
២. ធ្វើតាមដាន់ Automated Smoke Test Verification: ស្គ្រីប smoke-test.sh ត្រូវបានដំណើរការដើម្បីធ្វើតេស្តផ្លូវចរាចរណ៍ទិន្នន័យសំខាន់ៗទាំងអស់តាមរយៈ Nginx Reverse Proxy រួមមាន

/nginx-health, /, /api/health, និង /api/hits។ លទ្ធផលបញ្ជាក់ថាគ្រប់ Route ទាំងអស់ទទួលបាន 'PASS (HTTP 200)'។



រូបភាពទី ៦.២៖ *ការផ្ទៀងផ្ទាត់ Automated Smoke Test បញ្ជាក់ថាគ្រប់ Route ទទួលបាន PASS (HTTP 200)*

៣. ផ្ទៀងផ្ទាត់ Automated Quality Gate Enforcement: ដើម្បីបញ្ជាក់ពីប្រសិទ្ធភាពនៃ Quality Gate យើងបានធ្វើតេស្តដោយចេតនាបញ្ចូលកូដខុសទៅក្នុង Unit Test។ ជាលទ្ធផល Jenkins បានចាប់កំហុសភ្លាមៗនៅ Stage 'Test' និងបញ្ឈប់ Pipeline ជាបន្ទាន់ (Build Aborted) ដោយមិនអនុញ្ញាតឱ្យកូដដែលមានបញ្ហាត្រូវបាន Build ឬ Deploy ទៅកាន់ Production ឡើយ។



រូបភាពទី ៦.៣៖ *កស្តុតាងនៃ Automated Quality Gate: Pipeline បញ្ឈប់ភ្លាមៗនៅពេលមាន Test បរាជ័យ*

៧. ការពិភាក្សា និងការវិភាគស្ថាបត្យកម្ម

ផ្អែកលើការអនុវត្តជាក់ស្តែងក្នុងផ្នែកទី ៥ និងលទ្ធផលផ្ទៀងផ្ទាត់ក្នុងផ្នែកទី ៦ ក្រុមការងារបានធ្វើការវិភាគវាយតម្លៃលើចំណុចខ្លាំងនៃស្ថាបត្យកម្មប្រព័ន្ធ ព្រមទាំងដែនកំណត់បច្ចេកទេសដែលត្រូវពិចារណាសម្រាប់ការអភិវឌ្ឍទៅអនាគត។

៧.១- ចំណុចខ្លាំងនៃប្រព័ន្ធ (System Strengths)

- **ស្វ័យប្រវត្តិកម្ម GitOps ពេញលេញ (End-to-End GitOps):** លំហូរការងារត្រូវបានធ្វើស្វ័យប្រវត្តិកម្មទាំងស្រុងចាប់តាំងពី Developer ធ្វើការ Git Push រហូតដល់កូដត្រូវបានដាក់ឱ្យដំណើរការលើ AWS EC2 ដោយមិនត្រូវការការអន្តរាគមន៍ដោយដៃឡើយ
- **ភាពងាយស្រួលក្នុងការបង្កើតឡើងវិញ (100% Repeatability via IaC):** ហេដ្ឋារចនាសម្ព័ន្ធ Cloud ទាំងអស់ត្រូវបានសរសេរជាកូដ Terraform ដែលអនុញ្ញាតឱ្យយើងអាច

បង្កើតឡើងវិញ ឬបំផ្លាញចោល (Tear down) ក្នុងរយៈពេលក្រោម ៣ នាទីប្រកបដោយប្រសិទ្ធិភាព ១០០% ដោយគ្មានបញ្ហា Configuration Drift

- **សន្តិសុខបណ្តាញដាច់ដោយឡែក (Network Tier Isolation):** ការបែងចែកបណ្តាញ Overlay ជាពីរដាន់ (appnet សម្រាប់ Frontend/API និង datanet សម្រាប់ API/Redis) ជួយការពារទិន្នន័យក្នុង Redis មិនឱ្យមានការទាក់ទងដោយផ្ទាល់ពីខាងក្រៅ ឬពី Web Frontend ឡើយ
- **ភាពធន់ខ្ពស់ និងគ្មាន Downtime (Zero-Downtime Resilience):** ការកំណត់ 'order: start-first' ធានាថាអ្នកប្រើប្រាស់មិនជួបប្រទះការដាច់ចរាចរណ៍ទិន្នន័យ (Zero Dropped Packets) ក្នុងអំឡុងពេលដាក់កំណែកូដថ្មី។
- **ការធានាគុណភាពកូដម៉ត់ចត់ (Rigorous Quality Gates):** ការតេស្តកូដដោយស្វ័យប្រវត្តិតាមរយៈ flake8 និង pytest នៅក្នុង Container ដាច់ដោយឡែក ធានាថាកូដដែលគ្មានគុណភាព ឬមានកំហុសមិនអាចឆ្លងកាត់ទៅកាន់ Cluster បានឡើយ
- **ការតាមដានប្រតិបត្តិការដាក់ស្តែង (Real-Time ChatOps Telemetry):** ការផ្ញើសារជូនដំណឹង Telegram ស្វ័យប្រវត្តិជួយឱ្យក្រុមការងារដឹងពីស្ថានភាព Build និង Deploy បានភ្លាមៗលើទូរស័ព្ទដៃ។

៧.២- ចំណុចខ្សោយ និងដែនកំណត់ (Weaknesses & Architectural Limitations)

- **Single Point of Failure (SPOF) លើ Manager Node:** នៅក្នុងគម្រោងនេះ Swarm Cluster ត្រូវបានរៀបចំឡើងដោយមាន Manager Node តែមួយគត់ (១ Manager + ២ Workers)។ ប្រសិនបើ Manager Node នេះជួបឧប្បទ្ទវហេតុគាំង (Crash) ទោះបីជា Container នៅលើ Workers នៅតែបន្តដំណើរការក៏ដោយ ក៏យើងមិនអាចធ្វើការ Deploy ឬគ្រប់គ្រង Services បានទៀតនោះទេ រហូតដល់ Manager ត្រូវបានស្តារឡើងវិញ។ ដំណោះស្រាយសម្រាប់ Production គឺត្រូវប្រើ Manager ចំនួន ៣ ឬ ៥ Nodes ដើម្បីធានាបាននូវ Raft Consensus Quorum។
- **កង្វះដំណាក់កាលអនុម័តដោយដៃ (No Manual Approval Gate for Prod):** បច្ចុប្បន្ន Pipeline ដំណើរការ Deploy ទៅកាន់ Production ដោយស្វ័យប្រវត្តិបន្ទាប់ពី Test ជាប់។ សម្រាប់ប្រព័ន្ធសហគ្រាសធំៗ គួរមានដំណាក់កាល Manual Approval Gate សម្រាប់ឱ្យ Team Lead អនុម័តមុនពេល Deploy ទៅកាន់ Production។

- **ដែនកំណត់លើការត្រួតពិនិត្យស៊ីជម្រៅ (Advanced Metrics Monitoring):** ទោះបីជាប្រព័ន្ធមាន Telegram Alerts ក៏ដោយ ក៏ប្រព័ន្ធនៅមិនទាន់បានបំពាក់នូវប្រព័ន្ធប្រមូល Metrics និង Logging កម្រិតស៊ីជម្រៅដូចជា Prometheus និង Grafana ដើម្បីតាមដានកម្រិតប្រើប្រាស់ CPU, RAM និង Network Bandwidth នោះឡើយ។

៨. សេចក្តីសន្និដ្ឋាន និងអនុសាសន៍

គម្រោង Capstone Project នេះបានបង្ហាញយ៉ាងច្បាស់អំពីជោគជ័យនៃការរួមបញ្ចូលគ្នារវាងបច្ចេកវិទ្យា Git Version Control, Infrastructure as Code, Containerization, Multi-Node Orchestration, CI/CD Pipeline Automation, Cloud Security និង ChatOps Telemetry ឱ្យក្លាយទៅជាលំហូរការងារ GitOps តែមួយដែលមានលក្ខណៈរលូន និងអាចទុកចិត្តបាន។

តាមរយៈការអនុវត្តជាក់ស្តែង ក្រុមការងារទទួលបាននូវបទពិសោធន៍បច្ចេកទេសដ៏មានតម្លៃជាច្រើន៖

- **មេរៀនស្ថាបត្យកម្ម (Architecture Lesson):** ការចាកចេញពីប្រតិបត្តិការដោយដៃ (ClickOps) មកកាន់ Infrastructure as Code (Terraform) ជួយសន្សំពេលវេលា កាត់បន្ថយកំហុសមនុស្ស និងផ្តល់នូវ Audit Trail ពេញលេញលើធនធាន Cloud ទាំងអស់
- **មេរៀនភាពធន់ (Resilience Lesson):** ការប្រើប្រាស់ Docker Swarm Overlay Networks និង Ingress Routing Mesh ជួយសម្រួលដល់ការគ្រប់គ្រង Microservices និងការចែកចាយចរាចរណ៍ទិន្នន័យប្រកបដោយប្រសិទ្ធភាពខ្ពស់ ព្រមទាំងធានាការដាក់កំណែថ្មីដោយគ្មាន Downtime
- **មេរៀនប្រតិបត្តិការ (Operational Lesson):** ការដាក់បញ្ចូល Quality Gates ក្នុង Jenkinsfile ជួយទប់ស្កាត់កូដដែលមានបញ្ហាមិនឱ្យទៅដល់ដៃអ្នកប្រើប្រាស់ស្របពេលដែល Telegram ChatOps ផ្តល់នូវតម្លាភាព និងភាពងាយស្រួលក្នុងការតាមដានប្រតិបត្តិការ។

អនុសាសន៍សម្រាប់ការអភិវឌ្ឍទៅអនាគត (Recommendations for Future Work):

- **ពង្រីក High Availability:** ដំឡើង Multi-Manager Swarm Cluster (៣ Manager Nodes) ឆ្លងកាត់ Multiple AWS Availability Zones ដើម្បីលុបបំបាត់ Single Point of Failure

- **ពង្រឹងប្រព័ន្ធ Observability:** រួមបញ្ចូលឧបករណ៍ Prometheus និង Grafana សម្រាប់ Monitor ប្រព័ន្ធ និង ELK Stack (Elasticsearch, Logstash, Kibana) សម្រាប់ប្រមូលផ្តុំ Log ពី Container ទាំងអស់
- **បង្កើនកម្រិតសន្តិសុខ:** រួមបញ្ចូល HashiCorp Vault សម្រាប់គ្រប់គ្រង Dynamic Secrets និង Key Rotation ដោយស្វ័យប្រវត្តិ
- **ការធ្វើចំណាកស្រុកទៅ Kubernetes:** ពិចារណាផ្លាស់ប្តូរទៅកាន់ Kubernetes (Amazon EKS) នៅពេលប្រព័ន្ធមានសេវាកម្ម Microservices កើនឡើងដល់រាប់សិប និងត្រូវការ Auto-scaling កម្រិតខ្ពស់៕

ឯកសារយោង

Amazon Web Services. (n.d.). Amazon EC2 security groups for Linux instances. AWS Documentation. Retrieved November 3, 2026, from https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/ec2-security-groups.html

Amazon Web Services. (n.d.). Amazon Virtual Private Cloud (VPC) user guide. AWS Documentation. Retrieved November 3, 2026, from https://docs.aws.amazon.com/vpc/latest/userguide/what-is-amazon-vpc.html

Docker. (n.d.-a). Swarm mode overview and key concepts. Docker Documentation. Retrieved November 3, 2026, from https://docs.docker.com/engine/swarm/

Docker. (n.d.-b). Docker overview and container architecture. Docker Documentation. Retrieved November 3, 2026, from https://docs.docker.com/get-started/docker-overview/

Docker. (n.d.-c). Compose file reference and multi-container specification. Docker Documentation. Retrieved November 3, 2026, from https://docs.docker.com/compose/compose-file/

GitHub. (n.d.). About Git and GitHub documentation. GitHub Docs. Retrieved November 3, 2026, from https://docs.github.com/en/get-started/using-git/about-git

HashiCorp. (n.d.). Terraform architecture and HCL configuration documentation. HashiCorp Developer. Retrieved November 3, 2026, from https://developer.hashicorp.com/terraform/docs

Jenkins. (n.d.-a). Pipeline as code and Declarative Jenkinsfile syntax. Jenkins Documentation. Retrieved November 3, 2026, from https://www.jenkins.io/doc/book/pipeline/

Jenkins. (n.d.-b). Using credentials securely in Jenkins pipelines. Jenkins Documentation. Retrieved November 3, 2026, from https://www.jenkins.io/doc/book/using/using-credentials/

NGINX. (n.d.). NGINX reverse proxy and HTTP load balancing guide. NGINX Documentation. Retrieved November 3, 2026, from https://docs.nginx.com/nginx/admin-guide/web-server/reverse-proxy/

Python Software Foundation. (n.d.). Flask web framework and pytest testing framework documentation. Python Docs. Retrieved November 3, 2026, from https://docs.python.org/3/

Redis. (n.d.). Redis in-memory data store and Append Only File (AOF) persistence. Redis Documentation. Retrieved November 3, 2026, from https://redis.io/docs/